

1. Setup e Imports

```
import json as _json
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import pandas as pd
import subprocess
import matplotlib.pyplot as plt
import os
import sys

from tqdm import tqdm
from pathlib import Path
from torch.utils.data import Dataset, DataLoader

# Agregar path del proyecto para imports
sys.path.append(os.path.abspath('../..'))
from sae.models.sae import SparseAutoencoder
from sae.tools.naming_utils import save_checkpoint, get_model_name, get_extras_id, get_checkpoint_dir, get_experiment_dir, get_plots_dir, get_metrics_dir, get_logs_dir
from sae.tools.experiment_utils import get_experiment_dir, get_plots_dir, get_metrics_dir, get_logs_dir

# Configuración de visualización para Quarto/PDF
pd.set_option('display.width', 100)
pd.set_option('display.max_columns', None)
np.set_printoptions(linewidth=100, precision=4)
os.environ['COLUMNS'] = '100'

# Configuración de reproducibilidad
torch.manual_seed(42)
np.random.seed(42)
```

```
# Device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Print config con formato profesional
config_df = pd.DataFrame({
    'Configuración': ['Device', 'Seed', 'Output Width'],
    'Valor': [str(device), '42', '100']
})

print('\n' + '='*50)
print(' CONFIGURACIÓN INICIAL')
print('='*50)
print(config_df.to_string(index=False))
print('='*50)
```

```
=====
CONFIGURACIÓN INICIAL
=====
Configuración Valor
      Device  cuda
      Seed    42
Output Width  100
=====
```

2 Metadata para Naming de Checkpoints

```
# Metadata para naming de checkpoints (separada de hiperparámetros)
NAMING_META = {
    'variante': 'standard',          # Arquitectura del SAE
    'tecnica': 'p-annealing',        # P-annealing ( $L_p^p$  con p decreciente)
    'capa': 6,                       # Layer del modelo OthelloGPT
    'juegos': 25000,                 # Número de partidas
    'base_path': 'C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt', # Path base para cl
    'experiment_base_path': os.path.abspath('../../experiments') ,      # sae/experiments
    'extras': {
        'expansion': 16,
        'sparsity_coef': 5e-1,
        'epochs': 200,
        'early_stopping': False,
```

```

        'learning_rate': 1e-3,
        'p_start': 1.0,
        'p_end': 0.2,
        'n_sparsity_updates': 10,
        'anneal_start': 10000,
        'batch_size': 2048,
        'warmup_steps': 2000,
        'sparsity_warmup_steps': 4000,
        'estimated_epochs': 200
    }
}

print('\n Metadata de naming:')
for key, value in NAMING_META.items():
    print(f' {key}: {value}')

```

```

Metadata de naming:
variante: standard
tecnica: p-annealing
capa: 6
juegos: 25000
base_path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt
experiment_base_path: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments
extras: {'expansion': 16, 'sparsity_coef': 0.5, 'epochs': 200, 'early_stopping': False, 'l

```

```

extras_id = get_extras_id(
    get_checkpoint_dir(
        NAMING_META['base_path'],
        NAMING_META['variante'],
        NAMING_META['tecnica'],
        NAMING_META['capa'],
        NAMING_META['juegos']
    ),
    NAMING_META['extras']
) if NAMING_META.get('extras') else None

print(f'extras_id: {extras_id}')

```

```

extras_id: ex1

```

3. Configuración del SAE

Las activaciones ya fueron extraídas previamente con `sae/activations/run_extraction.py`. Aquí especificamos qué archivo cargar y los hiperparámetros del SAE.

```
# Configuración del SAE
CONFIG = {
    # Arquitectura
    'input_dim': 512,
    'hidden_dim': 8192,
    'tied_weights': False,

    # Entrenamiento
    'batch_size': 2048,
    'num_epochs': 200,
    'learning_rate': 1e-3,
    'warmup_steps': 2000,          # steps de LR warmup lineal
    'sparsity_warmup_steps': 4000, # steps de rampa de sparsity (0→1)
    'sparsity_coef': 5e-1,        # alpha inicial (loss sum(dim=-1).mean())

    # P-annealing
    'p_start': 1.0,               # p inicial (equivalente a L1)
    'p_end': 0.2,                 # p final
    'n_sparsity_updates': 10,     # número de saltos discretos
    'anneal_start': 10000,        # step en que empieza el annealing (después del warmup)
    'estimated_epochs': 200,      # épocas estimadas para condensar el schedule

    # Early Stopping
    'early_stopping': False,
    'patience': 50,              # (inactivo: early_stopping=False)

    # Datos
    'activations_file': Path(f"../../activations/data/layer{NAMING_META['capa']}_{NAMING_META['juegos']}.npy"),
    'save_dir': Path('./saved_model'),
    'plots_dir': get_plots_dir(
        NAMING_META['experiment_base_path'],
        NAMING_META['variante'],
        NAMING_META['tecnica'],
        NAMING_META['capa'],
        NAMING_META['juegos'],
        extras_id=extras_id
    )
}
```

```

# Crear directorio local del modelo
CONFIG['save_dir'].mkdir(parents=True, exist_ok=True)

# Mostrar configuración
config_data = {
    'Categoría': [],
    'Parámetro': [],
    'Valor': []
}

for key in ['input_dim', 'hidden_dim', 'tied_weights']:
    config_data['Categoría'].append('Arquitectura')
    config_data['Parámetro'].append(key)
    config_data['Valor'].append(str(CONFIG[key]))

for key in ['batch_size', 'num_epochs', 'learning_rate', 'warmup_steps', 'sparsity_warmup_steps']:
    config_data['Categoría'].append('Entrenamiento')
    config_data['Parámetro'].append(key)
    config_data['Valor'].append(str(CONFIG[key]))

for key in ['p_start', 'p_end', 'n_sparsity_updates', 'anneal_start', 'estimated_epochs']:
    config_data['Categoría'].append('P-Annealing')
    config_data['Parámetro'].append(key)
    config_data['Valor'].append(str(CONFIG[key]))

for key in ['early_stopping', 'patience']:
    config_data['Categoría'].append('Early Stopping')
    config_data['Parámetro'].append(key)
    config_data['Valor'].append(str(CONFIG[key]))

config_table = pd.DataFrame(config_data)

print('\n' + '='*70)
print(' CONFIGURACIÓN DEL SAE')
print('='*70)
print(config_table.to_string(index=False))
print('='*70)

print(f'\nArchivo de activaciones: {CONFIG["activations_file"]}')
print(f'Expansión: {CONFIG["hidden_dim"]/CONFIG["input_dim"]:.1f}x')
print(f'Plots dir: {CONFIG["plots_dir"]}')
print('='*70)

```

===== CONFIGURACIÓN DEL SAE =====

Categoría	Parámetro	Valor
Arquitectura	input_dim	512
Arquitectura	hidden_dim	8192
Arquitectura	tied_weights	False
Entrenamiento	batch_size	2048
Entrenamiento	num_epochs	200
Entrenamiento	learning_rate	0.001
Entrenamiento	warmup_steps	2000
Entrenamiento	sparsity_warmup_steps	4000
Entrenamiento	sparsity_coef	0.5
P-Annealing	p_start	1.0
P-Annealing	p_end	0.2
P-Annealing	n_sparsity_updates	10
P-Annealing	anneal_start	10000
P-Annealing	estimated_epochs	200
Early Stopping	early_stopping	False
Early Stopping	patience	50

=====

Archivo de activaciones: ..\..\..\activations\data\layer6_25000games.npy

Expansión: 16.0x

Plots dir: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard_annealing_standard\25000games\ex1\plots

=====

4. Dataset de Activaciones (Train/Validation Split 80-20%)

```
class ActivationsDataset(Dataset):
    """Dataset para cargar activaciones extraídas"""

    def __init__(self, activations_path):
        self.activations = np.load(activations_path)

    def __len__(self):
```

```

        return len(self.activations)

    def __getitem__(self, idx):
        return torch.tensor(self.activations[idx], dtype=torch.float32)

# Cargar dataset completo
full_dataset = ActivationsDataset(CONFIG['activations_file'])

# Información del dataset con formato profesional
dataset_info = pd.DataFrame({
    'Propiedad': ['Total muestras', 'Shape', 'Dtype', 'Tamaño en memoria'],
    'Valor': [
        f"{len(full_dataset):,}",
        str(full_dataset.activations.shape),
        str(full_dataset.activations.dtype),
        f"{full_dataset.activations.nbytes / 1024**2:.2f} MB"
    ]
})
print('\n' + '='*50)
print(' DATASET DE ACTIVACIONES')
print('='*50)
print(dataset_info.to_string(index=False))
print('='*50)

# Split train/validation (80-20)
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(
    full_dataset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(42)
)

# Resumen del split con formato profesional
split_info = pd.DataFrame({
    'Conjunto': ['Train', 'Validation', 'Total'],
    'Muestras': [
        f"{len(train_dataset):,}",
        f"{len(val_dataset):,}",
        f"{len(full_dataset):,}"
    ],
    'Porcentaje': [

```

```

        f"{len(train_dataset)/len(full_dataset):.1%}",
        f"{len(val_dataset)/len(full_dataset):.1%}",
        "100.0%"
    ]
})
print('\n' + '='*50)
print(' SPLIT DE DATOS (80-20)')
print('='*50)
print(split_info.to_string(index=False))
print('='*50)

# Crear DataLoaders
train_loader = DataLoader(
    train_dataset,
    batch_size=CONFIG['batch_size'],
    shuffle=True,
    num_workers=0,
    pin_memory=(device.type == 'cuda')
)

val_loader = DataLoader(
    val_dataset,
    batch_size=CONFIG['batch_size'],
    shuffle=False, # No shuffle para validación
    num_workers=0,
    pin_memory=(device.type == 'cuda')
)

# Resumen de DataLoaders
loader_info = pd.DataFrame({
    'DataLoader': ['Train', 'Validation'],
    'Batches': [len(train_loader), len(val_loader)],
    'Batch Size': [CONFIG['batch_size'], CONFIG['batch_size']],
    'Shuffle': ['Sí', 'No']
})

print('\n' + '='*50)
print(' DATALOADERS')
print('='*50)
print(loader_info.to_string(index=False))
print('='*50)

```



```
=====
DATASET DE ACTIVACIONES
=====

Propiedad      Valor
Total muestras  1,475,000
Shape (1475000, 512)
Dtype          float32
Tamaño en memoria  2880.86 MB
=====

=====
SPLIT DE DATOS (80-20)
=====

Conjunto  Muestras Porcentaje
Train 1,180,000      80.0%
Validation  295,000    20.0%
Total 1,475,000    100.0%
=====

=====
DATALOADERS
=====

DataLoader  Batches  Batch Size Shuffle
Train       577      2048      Sí
Validation   145      2048      No
=====
```

5. Modelo: Sparse Autoencoder

Modelo importado desde `sae.models.sae.SparseAutoencoder`

Arquitectura:

- **Encoder:** Linear $\rightarrow$ ReLU (sparsity natural)- **Decoder:** Linear (reconstrucción)

```
# Crear modelo desde el módulo
model = SparseAutoencoder(
    input_dim=CONFIG['input_dim'],
    hidden_dim=CONFIG['hidden_dim'],
    tied_weights=CONFIG['tied_weights']
).to(device)
```

```

# Calcular parámetros
num_params = sum(p.numel() for p in model.parameters())
encoder_params = sum(p.numel() for p in model.encoder.parameters())
if CONFIG['tied_weights']:
    decoder_params = CONFIG['input_dim'] # Solo bias
else:
    decoder_params = sum(p.numel() for p in model.decoder.parameters())

# Resumen del modelo con formato profesional
model_info = pd.DataFrame({
    'Componente': ['Input', 'Encoder', 'Hidden (Latent)', 'Decoder', 'Output', 'Total'],
    'Dimensión': [
        CONFIG['input_dim'],
        f"{CONFIG['input_dim']} → {CONFIG['hidden_dim']}",
        CONFIG['hidden_dim'],
        f"{CONFIG['hidden_dim']} → {CONFIG['input_dim']}",
        CONFIG['input_dim'],
        '-'
    ],
    'Parámetros': [
        '-',
        f"{encoder_params:,}",
        '-',
        f"{decoder_params:,}",
        '-',
        f"{num_params:,}"
    ]
})

print('\n' + '='*70)
print(' ARQUITECTURA DEL MODELO')
print('='*70)
print(model_info.to_string(index=False))
print('='*70)
print(f'\n Factor de expansión: {CONFIG["hidden_dim"]/CONFIG["input_dim"]:.1f}x')
print(f' Tied weights: {"Sí" if CONFIG["tied_weights"] else "No"}')
print(f' Device: {device}')
print('='*70)

```

```

=====
ARQUITECTURA DEL MODELO

```

```

=====
Componente  Dimensión  Parámetros
      Input           512          -
      Encoder 512 → 8192 4,202,496
Hidden (Latent)      8192          -
      Decoder 8192 → 512 4,194,304
      Output           512          -
      Total            - 8,397,312
=====

Factor de expansión: 16.0x
Tied weights: No
Device: cuda
=====

```

6. Función de Pérdida

Loss = MSE(reconstrucción) + λ × L1(activaciones)

- **MSE**: Mide qué tan bien reconstruimos las activaciones originales
- **L1**: Penaliza activaciones grandes → fuerza sparsity

```

class ConstrainedAdam(torch.optim.Adam):
    """
    Adam donde las columnas de decoder se proyectan a norma unitaria en cada step.
    Antes del paso: elimina la componente del gradiente paralela al decoder.
    Después del paso: renormaliza las columnas del decoder a norma 1.
    """
    def __init__(self, params, constrained_params, lr, betas=(0.9, 0.999)):
        super().__init__(params, lr=lr, betas=betas)
        self.constrained_params = list(constrained_params)

    def step(self, closure=None):
        with torch.no_grad():
            for p in self.constrained_params:
                normed_p = p / p.norm(dim=0, keepdim=True)
                p.grad -= (p.grad * normed_p).sum(dim=0, keepdim=True) * normed_p
        super().step(closure=closure)
        with torch.no_grad():
            for p in self.constrained_params:
                p /= p.norm(dim=0, keepdim=True)

```

```

def loss_function(x, reconstruction, hidden, sparsity_coeff, p, step=0, sparsity_warmup_steps=1000):
    """
    Pérdida compuesta: MSE (sum/mean) +  $L_p^p$  penalty con sparsity warmup

    Args:
        x: Activaciones originales
        reconstruction: Activaciones reconstruidas
        hidden: Activaciones latentes (sparse,  $\geq 0$  por ReLU)
        sparsity_coeff: Peso adaptativo de la penalización (alpha)
        p: Exponente actual del schedule de annealing
        step: Step global actual (para sparsity warmup)
        sparsity_warmup_steps: Steps de rampa de sparsity (0 = desactivado)
    """
    # BUG2 FIX: sum sobre features luego mean sobre batch (igual que referencia)
    mse_loss = (reconstruction - x).pow(2).sum(dim=-1).mean()
    # BUG4 FIX: sparsity warmup - rampa lineal de 0 a 1
    sparsity_scale = min(step / sparsity_warmup_steps, 1.0) if sparsity_warmup_steps > 0 else 1.0
    sparsity_loss = hidden.pow(p).sum(dim=1).mean() #  $L_p^p$ :  $\sum |f_i|^p$ 
    total_loss = mse_loss + sparsity_coeff * sparsity_scale * sparsity_loss
    return total_loss, mse_loss, sparsity_loss

# ConstrainedAdam: pasa model.decoder.parameters() como parámetros constrained
optimizer = ConstrainedAdam(
    model.parameters(),
    model.decoder.parameters(),
    lr=CONFIG['learning_rate']
)

# BUG3 FIX: LR warmup lineal - sube de 0 a lr en warmup_steps
def _lr_lambda(step):
    warmup = CONFIG.get('warmup_steps', 1000)
    return min(step / warmup, 1.0) if warmup > 0 else 1.0

scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda=_lr_lambda)

optim_info = pd.DataFrame({
    'Parámetro': ['Optimizador', 'Learning Rate', 'Sparsity Coef (inicial)', 'p_start', 'p_end'],
    'Valor': [
        'ConstrainedAdam',
        f"{CONFIG['learning_rate']:.0e}",
        f"{CONFIG['sparsity_coef']:.0e}",
        0,
        1
    ]
})

```

```

        f"{CONFIG['p_start']}",
        f"{CONFIG['p_end']}",
        f"{CONFIG['n_sparsity_updates']}",
        f"{CONFIG['anneal_start']}"
    ]
})

print('\n' + '='*55)
print(' OPTIMIZACIÓN')
print('='*55)
print(optim_info.to_string(index=False))
print('='*55)

```

```

=====
OPTIMIZACIÓN
=====

```

Parámetro	Valor
Optimizador	ConstrainedAdam
Learning Rate	1e-03
Sparsity Coef (inicial)	5e-01
p_start	1.0
p_end	0.2
n_sparsity_updates	10
anneal_start	10000

```

=====

```

7. Loop de Entrenamiento

```

# Historial de entrenamiento
history = {
    'train_total_loss': [],
    'train_mse_loss': [],
    'train_sparsity_loss': [],
    'train_l0_sparsity': [],
    'train_fraction_active': [],
    'val_total_loss': [],
    'val_mse_loss': [],
    'val_sparsity_loss': [],
    'val_l0_sparsity': [],

```

```

    'val_fraction_active': [],
    'sparsity_coef_history': [],      # snapshot de sparsity_coef al final de cada época
    'p_history': [],                  # snapshot de p al final de cada época
}

# Early stopping variables
best_val_mse = float('inf')
best_epoch = 0
epochs_no_improve = 0

# ===== P-ANNEALING STATE =====
p = CONFIG['p_start']
next_p = None
p_step_count = 0
sparsity_coef = CONFIG['sparsity_coef']
sparsity_queue = []
sparsity_queue_length = 10

total_steps = CONFIG['num_epochs'] * len(train_loader)
# BUG5 FIX: usar expected_steps (no total_steps) para que el schedule
# de annealing complete antes del early stopping
estimated_epochs = CONFIG.get('estimated_epochs', 50)
expected_steps = estimated_epochs * len(train_loader)
_update_steps_tensor = torch.linspace(
    CONFIG['anneal_start'], expected_steps, CONFIG['n_sparsity_updates'], dtype=torch.int
)
sparsity_update_steps_set = set(_update_steps_tensor.tolist())
sparsity_update_steps_list = sorted(sparsity_update_steps_set)
p_values = torch.linspace(CONFIG['p_start'], CONFIG['p_end'], CONFIG['n_sparsity_updates']).tolist()
global_step = 0

print(f'\n Iniciando entrenamiento: {CONFIG["num_epochs"]} épocas máximo')
print(f' P-annealing: p {CONFIG["p_start"]} → {CONFIG["p_end"]} en {CONFIG["n_sparsity_updates"]} épocas')
print(f' Early stopping: patience={CONFIG["patience"]} épocas')
print('=' * 70)

for epoch in range(CONFIG['num_epochs']):

    # ===== ENTRENAMIENTO =====
    model.train()

    epoch_total_loss = 0

```

```

epoch_mse_loss = 0
epoch_sparsity_loss = 0
epoch_l0 = 0
epoch_fraction_active = 0

pbar = tqdm(train_loader, desc=f'Época {epoch+1}/{CONFIG["num_epochs"]} [Train] p={p:.3f}')

for batch in pbar:
    x = batch.to(device)

    # Forward pass
    reconstruction, hidden = model(x)

    # ---- P-ANNEALING: lookahead para la cola ----
    with torch.no_grad():
        if next_p is not None:
            lp_curr = hidden.pow(p).sum(dim=1).mean().item()
            lp_next = hidden.pow(next_p).sum(dim=1).mean().item()
            sparsity_queue.append([lp_curr, lp_next])
            sparsity_queue = sparsity_queue[-sparsity_queue_length:]

    # Verificar si este step es un punto de actualización de p
    if (global_step in sparsity_update_steps_set
        and p_step_count < CONFIG['n_sparsity_updates']
        and global_step >= sparsity_update_steps_list[p_step_count]):
        # Re-escalar sparsity_coeff para compensar el cambio de magnitud de  $L_p^p$ 
        if next_p is not None and len(sparsity_queue) > 0:
            local_curr = sum(i[0] for i in sparsity_queue) / len(sparsity_queue)
            local_next = sum(i[1] for i in sparsity_queue) / len(sparsity_queue)
            if local_next > 0:
                sparsity_coeff = sparsity_coeff * (local_curr / local_next)
        # Actualizar p al siguiente valor del schedule
        p = p_values[p_step_count]
        if p_step_count < CONFIG['n_sparsity_updates'] - 1:
            next_p = p_values[p_step_count + 1]
        else:
            next_p = CONFIG['p_end']
        p_step_count += 1

    # Calcular pérdida con p actual
    total_loss, mse_loss, sparsity_loss = loss_function(
        x, reconstruction, hidden, sparsity_coeff, p,

```

```

        step=global_step,
        sparsity_warmup_steps=CONFIG.get('sparsity_warmup_steps', 0)
    )

    # Backward pass
    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()
    scheduler.step() # BUG3 FIX: LR warmup step

    global_step += 1

    # Métricas
    with torch.no_grad():
        l0 = (hidden > 0).float().sum(dim=1).mean().item()
        fraction_active = (hidden > 0).float().mean().item()

    epoch_total_loss += total_loss.item()
    epoch_mse_loss += mse_loss.item()
    epoch_sparsity_loss += sparsity_loss.item()
    epoch_l0 += l0
    epoch_fraction_active += fraction_active

    pbar.set_postfix({
        'loss': f'{total_loss.item():.4f}',
        'mse': f'{mse_loss.item():.4f}',
        'l0': f'{l0:.1f}',
        'p': f'{p:.3f}'
    })

    # Promedios de entrenamiento
    num_train_batches = len(train_loader)
    avg_train_total_loss = epoch_total_loss / num_train_batches
    avg_train_mse_loss = epoch_mse_loss / num_train_batches
    avg_train_sparsity_loss = epoch_sparsity_loss / num_train_batches
    avg_train_l0 = epoch_l0 / num_train_batches
    avg_train_fraction_active = epoch_fraction_active / num_train_batches

    # ===== VALIDACIÓN =====
    model.eval()

    val_total_loss = 0

```



```

val_mse_loss = 0
val_sparsity_loss = 0
val_l0 = 0
val_fraction_active = 0

with torch.no_grad():
    for batch in val_loader:
        x = batch.to(device)
        reconstruction, hidden = model(x)
        total_loss, mse_loss, sparsity_loss = loss_function(
            x, reconstruction, hidden, sparsity_coeff, p,
            step=global_step,
            sparsity_warmup_steps=CONFIG.get('sparsity_warmup_steps', 0)
        )
        l0 = (hidden > 0).float().sum(dim=1).mean().item()
        fraction_active = (hidden > 0).float().mean().item()

        val_total_loss += total_loss.item()
        val_mse_loss += mse_loss.item()
        val_sparsity_loss += sparsity_loss.item()
        val_l0 += l0
        val_fraction_active += fraction_active

num_val_batches = len(val_loader)
avg_val_total_loss = val_total_loss / num_val_batches
avg_val_mse_loss = val_mse_loss / num_val_batches
avg_val_sparsity_loss = val_sparsity_loss / num_val_batches
avg_val_l0 = val_l0 / num_val_batches
avg_val_fraction_active = val_fraction_active / num_val_batches

# Guardar en historial
history['train_total_loss'].append(avg_train_total_loss)
history['train_mse_loss'].append(avg_train_mse_loss)
history['train_sparsity_loss'].append(avg_train_sparsity_loss)
history['train_l0_sparsity'].append(avg_train_l0)
history['train_fraction_active'].append(avg_train_fraction_active)

history['val_total_loss'].append(avg_val_total_loss)
history['val_mse_loss'].append(avg_val_mse_loss)
history['val_sparsity_loss'].append(avg_val_sparsity_loss)
history['val_l0_sparsity'].append(avg_val_l0)
history['val_fraction_active'].append(avg_val_fraction_active)

```

```

history['sparsity_coef_history'].append(sparsity_coef)
history['p_history'].append(p)

# ===== EARLY STOPPING =====
if CONFIG['early_stopping']:
    if avg_val_mse_loss < best_val_mse:
        best_val_mse = avg_val_mse_loss
        best_epoch = epoch + 1
        epochs_no_improve = 0

    best_checkpoint = {
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'config': CONFIG,
        'history': history,
        'epoch': best_epoch,
        'val_mse': best_val_mse
    }

    best_model_name = get_model_name(
        variante=NAMING_META['variante'],
        tecnica=NAMING_META['tecnica'],
        capa=NAMING_META['capa'],
        juegos=NAMING_META['juegos'],
        estado='best'
    )
    torch.save(best_checkpoint, CONFIG['save_dir'] / best_model_name)

save_checkpoint(
    model,
    base_path=NAMING_META['base_path'],
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='best',
    extras=NAMING_META['extras'],
    config=CONFIG,
    history=history,
    optimizer=optimizer
)

```

```

        print(f' Mejor modelo guardado en época {best_epoch} (Val MSE: {best_val_mse:.4f}')
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= CONFIG['patience']:
            print(f'\n Early stopping activado en época {epoch+1}')
            print(f' Mejor modelo: época {best_epoch} con Val MSE = {best_val_mse:.4f}')
            break

# Log cada 5 épocas
if (epoch + 1) % 5 == 0 or epoch == 0:
    print(f'\n Época {epoch+1}/{CONFIG["num_epochs"]} | p: {p:.3f} | : {sparsity_coeff:.3f}')
    print(f' Train - Total: {avg_train_total_loss:.4f} | MSE: {avg_train_mse_loss:.4f}')
    print(f' Val - Total: {avg_val_total_loss:.4f} | MSE: {avg_val_mse_loss:.4f} | L0: {l0_sparsity:.3f}')
    if CONFIG['early_stopping']:
        print(f' Best Val MSE: {best_val_mse:.4f} @ época {best_epoch} | Sin mejora: {epochs_no_improve}')

print('\n' + '='*70)
print(' RESUMEN DEL ENTRENAMIENTO')
print('='*70)

if CONFIG['early_stopping']:
    print(f'Mejor modelo en época {best_epoch} con Val MSE = {best_val_mse:.4f}')
    print(f'Early stopping tras {len(history["train_total_loss"])} épocas')
else:
    print(f'Entrenamiento completo: {CONFIG["num_epochs"]} épocas')

final_metrics = pd.DataFrame({
    'Métrica': ['Total Loss', 'MSE Loss', 'L0 Sparsity', 'Fraction Active'],
    'Train': [
        f"{history['train_total_loss'][-1]:.4f}",
        f"{history['train_mse_loss'][-1]:.4f}",
        f"{history['train_l0_sparsity'][-1]:.1f}",
        f"{history['train_fraction_active'][-1]:.2%}"
    ],
    'Validation': [
        f"{history['val_total_loss'][-1]:.4f}",
        f"{history['val_mse_loss'][-1]:.4f}",
        f"{history['val_l0_sparsity'][-1]:.1f}",
        f"{history['val_fraction_active'][-1]:.2%}"
    ]
})

```

```
print('\nMétricas Finales (Última Época):')
print(final_metrics.to_string(index=False))
print('='*70)
```

Iniciando entrenamiento: 200 épocas máximo
P-annealing: p 1.0 → 0.2 en 10 saltos
Early stopping: patience=50 épocas

```
=====
```

Época 1/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:21<00:00, 26.56it/s, loss=26.66]

Época 1/200 | p: 1.000 | : 5.00e-01
Train - Total: 20.9328 | MSE: 8.4851 | L0: 4601.6
Val - Total: 26.6619 | MSE: 5.7655 | L0: 3845.9

Época 2/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.41it/s, loss=39.15]
Época 3/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.62it/s, loss=49.15]
Época 4/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.56it/s, loss=58.15]
Época 5/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.65it/s, loss=68.15]

Época 5/200 | p: 1.000 | : 5.00e-01
Train - Total: 64.1515 | MSE: 19.1559 | L0: 1072.9
Val - Total: 69.0967 | MSE: 21.9333 | L0: 964.7

Época 6/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.56it/s, loss=75.15]
Época 7/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.77it/s, loss=85.15]
Época 8/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.62it/s, loss=84.15]
Época 9/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.68it/s, loss=85.15]
Época 10/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.52it/s, loss=84.15]

Época 10/200 | p: 1.000 | : 5.00e-01
Train - Total: 83.5367 | MSE: 29.8043 | L0: 636.0
Val - Total: 84.2720 | MSE: 30.9007 | L0: 634.5

Época 11/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.73it/s, loss=85
 Época 12/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.28it/s, loss=83
 Época 13/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.77it/s, loss=83
 Época 14/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.69it/s, loss=83
 Época 15/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.73it/s, loss=82

Época 15/200 | p: 1.000 | : 5.00e-01
 Train - Total: 82.4540 | MSE: 29.8569 | L0: 604.1
 Val - Total: 83.4329 | MSE: 30.9043 | L0: 606.8

Época 16/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.71it/s, loss=80
 Época 17/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.58it/s, loss=82
 Época 18/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 28.09it/s, loss=83
 Época 19/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.64it/s, loss=83
 Época 20/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.68it/s, loss=82

Época 20/200 | p: 1.000 | : 5.00e-01
 Train - Total: 82.0766 | MSE: 30.1480 | L0: 579.7
 Val - Total: 83.1353 | MSE: 31.3745 | L0: 582.2

Época 21/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.59it/s, loss=82
 Época 22/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.87it/s, loss=83
 Época 23/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.72it/s, loss=82
 Época 24/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.79it/s, loss=82
 Época 25/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.71it/s, loss=80

Época 25/200 | p: 1.000 | : 5.00e-01
 Train - Total: 81.8749 | MSE: 30.2793 | L0: 568.5
 Val - Total: 82.9964 | MSE: 31.5735 | L0: 571.4

Época 26/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.77it/s, loss=82
 Época 27/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.75it/s, loss=83
 Época 28/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.78it/s, loss=82
 Época 29/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.63it/s, loss=82
 Época 30/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:21<00:00, 27.43it/s, loss=82

Época 30/200 | p: 1.000 | : 5.00e-01
Train - Total: 81.7201 | MSE: 30.3086 | L0: 563.2
Val - Total: 82.8880 | MSE: 31.6929 | L0: 565.8

Época 31/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.72it/s, loss=83
Época 32/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.84it/s, loss=83
Época 33/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.77it/s, loss=83
Época 34/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.66it/s, loss=83
Época 35/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.77it/s, loss=83

Época 35/200 | p: 1.000 | : 5.00e-01
Train - Total: 81.6009 | MSE: 30.3150 | L0: 559.9
Val - Total: 82.8338 | MSE: 31.6809 | L0: 563.6

Época 36/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.86it/s, loss=83
Época 37/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.76it/s, loss=83
Época 38/200 [Train] p=1.000 =5.00e-01: 100%| | 577/577 [00:20<00:00, 27.49it/s, loss=82
Época 39/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:21<00:00, 27.13it/s, loss=84
Época 40/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:20<00:00, 27.50it/s, loss=83

Época 40/200 | p: 0.911 | : 4.50e-01
Train - Total: 81.6489 | MSE: 32.4065 | L0: 479.2
Val - Total: 83.0697 | MSE: 34.1263 | L0: 477.3

Época 41/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:20<00:00, 27.49it/s, loss=83
Época 42/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:21<00:00, 27.41it/s, loss=83
Época 43/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:21<00:00, 27.47it/s, loss=82
Época 44/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:21<00:00, 27.47it/s, loss=83
Época 45/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:20<00:00, 27.51it/s, loss=82

Época 45/200 | p: 0.911 | : 4.50e-01
Train - Total: 81.6284 | MSE: 32.9476 | L0: 457.4
Val - Total: 83.1126 | MSE: 34.7433 | L0: 458.4

Época 46/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:21<00:00, 27.44it/s, loss=83
 Época 47/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:21<00:00, 27.32it/s, loss=83
 Época 48/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:20<00:00, 27.53it/s, loss=83
 Época 49/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:21<00:00, 27.31it/s, loss=82
 Época 50/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:21<00:00, 27.26it/s, loss=83

Época 50/200 | p: 0.911 | : 4.50e-01
 Train - Total: 81.5826 | MSE: 33.0641 | L0: 451.5
 Val - Total: 83.1154 | MSE: 34.7814 | L0: 454.2

Época 51/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:21<00:00, 27.40it/s, loss=83
 Época 52/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:20<00:00, 27.50it/s, loss=83
 Época 53/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:20<00:00, 27.52it/s, loss=83
 Época 54/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:20<00:00, 27.52it/s, loss=79
 Época 55/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:21<00:00, 27.47it/s, loss=82

Época 55/200 | p: 0.911 | : 4.50e-01
 Train - Total: 81.5269 | MSE: 33.0933 | L0: 448.8
 Val - Total: 83.0602 | MSE: 34.9231 | L0: 450.3

Época 56/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:20<00:00, 27.56it/s, loss=82
 Época 57/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:20<00:00, 27.58it/s, loss=83
 Época 58/200 [Train] p=0.911 =4.50e-01: 100%| | 577/577 [00:20<00:00, 27.60it/s, loss=82
 Época 59/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.53it/s, loss=82
 Época 60/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.69it/s, loss=80

Época 60/200 | p: 0.822 | : 4.09e-01
 Train - Total: 81.5863 | MSE: 34.9524 | L0: 396.5
 Val - Total: 83.3217 | MSE: 37.0322 | L0: 394.3

Época 61/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.70it/s, loss=83
 Época 62/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.58it/s, loss=80
 Época 63/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.67it/s, loss=83
 Época 64/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.69it/s, loss=79
 Época 65/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.68it/s, loss=83

Época 65/200 | p: 0.822 | : 4.09e-01
Train - Total: 81.5814 | MSE: 35.4949 | L0: 378.2
Val - Total: 83.4105 | MSE: 37.6789 | L0: 378.5

Época 66/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.51it/s, loss=79
Época 67/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:21<00:00, 27.47it/s, loss=82
Época 68/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:21<00:00, 27.45it/s, loss=83
Época 69/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.50it/s, loss=82
Época 70/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:21<00:00, 27.35it/s, loss=80

Época 70/200 | p: 0.822 | : 4.09e-01
Train - Total: 81.5416 | MSE: 35.6037 | L0: 373.0
Val - Total: 83.4164 | MSE: 37.7314 | L0: 374.5

Época 71/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.49it/s, loss=83
Época 72/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.48it/s, loss=82
Época 73/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:21<00:00, 27.40it/s, loss=82
Época 74/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:21<00:00, 27.37it/s, loss=83
Época 75/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:21<00:00, 27.43it/s, loss=83

Época 75/200 | p: 0.822 | : 4.09e-01
Train - Total: 81.5070 | MSE: 35.6326 | L0: 370.4
Val - Total: 83.4041 | MSE: 37.8846 | L0: 371.3

Época 76/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:21<00:00, 27.39it/s, loss=83
Época 77/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:21<00:00, 27.39it/s, loss=80
Época 78/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:21<00:00, 27.45it/s, loss=82
Época 79/200 [Train] p=0.822 =4.09e-01: 100%| | 577/577 [00:20<00:00, 27.48it/s, loss=84
Época 80/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:21<00:00, 27.45it/s, loss=82

Época 80/200 | p: 0.733 | : 3.74e-01
Train - Total: 81.6815 | MSE: 37.3266 | L0: 335.2
Val - Total: 83.6782 | MSE: 39.7637 | L0: 332.3

Época 81/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:21<00:00, 27.44it/s, loss=8
 Época 82/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.48it/s, loss=8
 Época 83/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.55it/s, loss=8
 Época 84/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.51it/s, loss=8
 Época 85/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.54it/s, loss=8

Época 85/200 | p: 0.733 | : 3.74e-01
 Train - Total: 81.5889 | MSE: 37.7445 | L0: 319.8
 Val - Total: 83.7090 | MSE: 40.3140 | L0: 319.3

Época 86/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.67it/s, loss=8
 Época 87/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:21<00:00, 27.47it/s, loss=8
 Época 88/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.51it/s, loss=8
 Época 89/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.48it/s, loss=8
 Época 90/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.65it/s, loss=8

Época 90/200 | p: 0.733 | : 3.74e-01
 Train - Total: 81.5490 | MSE: 37.7949 | L0: 315.1
 Val - Total: 83.6884 | MSE: 40.3671 | L0: 314.9

Época 91/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.75it/s, loss=8
 Época 92/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.64it/s, loss=8
 Época 93/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.51it/s, loss=8
 Época 94/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.52it/s, loss=8
 Época 95/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.58it/s, loss=8

Época 95/200 | p: 0.733 | : 3.74e-01
 Train - Total: 81.4910 | MSE: 37.7489 | L0: 312.5
 Val - Total: 83.6747 | MSE: 40.2581 | L0: 313.1

Época 96/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.56it/s, loss=8
 Época 97/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.70it/s, loss=8
 Época 98/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.63it/s, loss=8
 Época 99/200 [Train] p=0.733 =3.74e-01: 100%| | 577/577 [00:20<00:00, 27.64it/s, loss=8
 Época 100/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.69it/s, loss=8

Época 100/200 | p: 0.644 | : 3.44e-01
Train - Total: 81.8200 | MSE: 39.2691 | L0: 288.2
Val - Total: 83.9421 | MSE: 41.8641 | L0: 284.7

Época 101/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.61it/s, loss=8
Época 102/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.70it/s, loss=8
Época 103/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.65it/s, loss=7
Época 104/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.52it/s, loss=8
Época 105/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.50it/s, loss=8

Época 105/200 | p: 0.644 | : 3.44e-01
Train - Total: 81.5158 | MSE: 39.1068 | L0: 274.8
Val - Total: 83.7348 | MSE: 41.7054 | L0: 274.2

Época 106/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.57it/s, loss=8
Época 107/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.65it/s, loss=8
Época 108/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.50it/s, loss=7
Época 109/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.67it/s, loss=8
Época 110/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.67it/s, loss=8

Época 110/200 | p: 0.644 | : 3.44e-01
Train - Total: 81.3119 | MSE: 38.3602 | L0: 271.6
Val - Total: 83.4943 | MSE: 40.7966 | L0: 271.8

Época 111/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.64it/s, loss=8
Época 112/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.54it/s, loss=7
Época 113/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.58it/s, loss=8
Época 114/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.66it/s, loss=8
Época 115/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.77it/s, loss=8

Época 115/200 | p: 0.644 | : 3.44e-01
Train - Total: 81.0867 | MSE: 37.5683 | L0: 269.9
Val - Total: 83.2568 | MSE: 40.1090 | L0: 269.3

Época 116/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.78it/s, loss=8
 Época 117/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.65it/s, loss=8
 Época 118/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.63it/s, loss=8
 Época 119/200 [Train] p=0.644 =3.44e-01: 100%| | 577/577 [00:20<00:00, 27.69it/s, loss=8
 Época 120/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.72it/s, loss=8

Época 120/200 | p: 0.556 | : 3.21e-01
 Train - Total: 81.7060 | MSE: 38.7827 | L0: 253.1
 Val - Total: 83.7107 | MSE: 41.2366 | L0: 249.1

Época 121/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.66it/s, loss=7
 Época 122/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.65it/s, loss=8
 Época 123/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.66it/s, loss=8
 Época 124/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.68it/s, loss=8
 Época 125/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:21<00:00, 27.42it/s, loss=8

Época 125/200 | p: 0.556 | : 3.21e-01
 Train - Total: 81.0439 | MSE: 38.0544 | L0: 242.0
 Val - Total: 83.1909 | MSE: 40.4568 | L0: 241.6

Época 126/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.70it/s, loss=7
 Época 127/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.71it/s, loss=8
 Época 128/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.74it/s, loss=8
 Época 129/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.72it/s, loss=8
 Época 130/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.65it/s, loss=8

Época 130/200 | p: 0.556 | : 3.21e-01
 Train - Total: 80.7727 | MSE: 37.4014 | L0: 240.1
 Val - Total: 82.9704 | MSE: 39.9687 | L0: 239.2

Época 131/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.67it/s, loss=7
 Época 132/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.64it/s, loss=7
 Época 133/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.65it/s, loss=8
 Época 134/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.61it/s, loss=8
 Época 135/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.64it/s, loss=7

Época 135/200 | p: 0.556 | : 3.21e-01
 Train - Total: 80.5567 | MSE: 36.9272 | L0: 239.0
 Val - Total: 82.7892 | MSE: 39.4691 | L0: 238.7

Época 136/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.69it/s, loss=7
 Época 137/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.63it/s, loss=7
 Época 138/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.70it/s, loss=8
 Época 139/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.61it/s, loss=8
 Época 140/200 [Train] p=0.556 =3.21e-01: 100%| | 577/577 [00:20<00:00, 27.68it/s, loss=8

Época 140/200 | p: 0.467 | : 3.03e-01
 Train - Total: 81.5958 | MSE: 38.6248 | L0: 229.2
 Val - Total: 83.9811 | MSE: 41.6940 | L0: 223.7

Época 141/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.56it/s, loss=8
 Época 142/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.67it/s, loss=8
 Época 143/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.51it/s, loss=8
 Época 144/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.64it/s, loss=8
 Época 145/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.68it/s, loss=8

Época 145/200 | p: 0.467 | : 3.03e-01
 Train - Total: 81.3530 | MSE: 39.1842 | L0: 216.9
 Val - Total: 83.5407 | MSE: 41.6797 | L0: 216.1

Época 146/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.65it/s, loss=8
 Época 147/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.68it/s, loss=7
 Época 148/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.64it/s, loss=8
 Época 149/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:21<00:00, 27.42it/s, loss=7
 Época 150/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.50it/s, loss=8

Época 150/200 | p: 0.467 | : 3.03e-01
 Train - Total: 81.1531 | MSE: 38.8627 | L0: 215.6
 Val - Total: 83.4574 | MSE: 41.4334 | L0: 215.3

Época 151/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.69it/s, loss=8
 Época 152/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.67it/s, loss=8
 Época 153/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:20<00:00, 27.64it/s, loss=8
 Época 154/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:21<00:00, 26.88it/s, loss=8
 Época 155/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:21<00:00, 27.33it/s, loss=8

Época 155/200 | p: 0.467 | : 3.03e-01
 Train - Total: 81.0427 | MSE: 38.6383 | L0: 215.0
 Val - Total: 83.3896 | MSE: 41.3960 | L0: 213.9

Época 156/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:21<00:00, 27.38it/s, loss=8
 Época 157/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:21<00:00, 27.36it/s, loss=8
 Época 158/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:21<00:00, 27.44it/s, loss=8
 Época 159/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:21<00:00, 27.28it/s, loss=8
 Época 160/200 [Train] p=0.467 =3.03e-01: 100%| | 577/577 [00:21<00:00, 27.37it/s, loss=8

Época 160/200 | p: 0.378 | : 2.86e-01
 Train - Total: 81.8625 | MSE: 39.7943 | L0: 210.9
 Val - Total: 85.2860 | MSE: 44.2570 | L0: 203.8

Época 161/200 [Train] p=0.378 =2.86e-01: 100%| | 577/577 [00:21<00:00, 27.46it/s, loss=8
 Época 162/200 [Train] p=0.378 =2.86e-01: 100%| | 577/577 [00:21<00:00, 27.37it/s, loss=8
 Época 163/200 [Train] p=0.378 =2.86e-01: 100%| | 577/577 [00:21<00:00, 27.28it/s, loss=8
 Época 164/200 [Train] p=0.378 =2.86e-01: 100%| | 577/577 [00:20<00:00, 27.49it/s, loss=8
 Época 165/200 [Train] p=0.378 =2.86e-01: 100%| | 577/577 [00:21<00:00, 27.35it/s, loss=8

Época 165/200 | p: 0.378 | : 2.86e-01
 Train - Total: 82.7116 | MSE: 42.0737 | L0: 197.6
 Val - Total: 85.0396 | MSE: 44.7483 | L0: 196.4

Época 166/200 [Train] p=0.378 =2.86e-01: 100%| | 577/577 [00:20<00:00, 27.62it/s, loss=8
 Época 167/200 [Train] p=0.378 =2.86e-01: 100%| | 577/577 [00:20<00:00, 27.52it/s, loss=8
 Época 168/200 [Train] p=0.378 =2.86e-01: 100%| | 577/577 [00:20<00:00, 27.65it/s, loss=8
 Época 169/200 [Train] p=0.378 =2.86e-01: 100%| | 577/577 [00:20<00:00, 27.58it/s, loss=8
 Época 170/200 [Train] p=0.378 =2.86e-01: 100%| | 577/577 [00:20<00:00, 27.52it/s, loss=8

Época 170/200 | p: 0.378 | : 2.86e-01
Train - Total: 82.5615 | MSE: 42.0041 | L0: 195.9
Val - Total: 84.9010 | MSE: 44.7388 | L0: 194.6

Época 171/200 [Train]	p=0.378	=2.86e-01: 100%	577/577 [00:20<00:00, 27.52it/s, loss=8
Época 172/200 [Train]	p=0.378	=2.86e-01: 100%	577/577 [00:20<00:00, 27.54it/s, loss=8
Época 173/200 [Train]	p=0.378	=2.86e-01: 100%	577/577 [00:20<00:00, 27.61it/s, loss=8
Época 174/200 [Train]	p=0.378	=2.86e-01: 100%	577/577 [00:20<00:00, 27.57it/s, loss=8
Época 175/200 [Train]	p=0.378	=2.86e-01: 100%	577/577 [00:20<00:00, 27.72it/s, loss=8

Época 175/200 | p: 0.378 | : 2.86e-01
Train - Total: 82.5326 | MSE: 42.0040 | L0: 195.2
Val - Total: 84.8593 | MSE: 44.6821 | L0: 194.0

Época 176/200 [Train]	p=0.378	=2.86e-01: 100%	577/577 [00:20<00:00, 27.69it/s, loss=8
Época 177/200 [Train]	p=0.378	=2.86e-01: 100%	577/577 [00:20<00:00, 27.59it/s, loss=8
Época 178/200 [Train]	p=0.378	=2.86e-01: 100%	577/577 [00:20<00:00, 27.61it/s, loss=8
Época 179/200 [Train]	p=0.378	=2.86e-01: 100%	577/577 [00:20<00:00, 27.58it/s, loss=8
Época 180/200 [Train]	p=0.378	=2.86e-01: 100%	577/577 [00:20<00:00, 27.52it/s, loss=8

Época 180/200 | p: 0.289 | : 2.70e-01
Train - Total: 82.8510 | MSE: 42.3608 | L0: 194.2
Val - Total: 86.8008 | MSE: 47.2175 | L0: 188.7

Época 181/200 [Train]	p=0.289	=2.70e-01: 100%	577/577 [00:20<00:00, 27.63it/s, loss=8
Época 182/200 [Train]	p=0.289	=2.70e-01: 100%	577/577 [00:20<00:00, 27.64it/s, loss=8
Época 183/200 [Train]	p=0.289	=2.70e-01: 100%	577/577 [00:20<00:00, 27.74it/s, loss=8
Época 184/200 [Train]	p=0.289	=2.70e-01: 100%	577/577 [00:20<00:00, 27.75it/s, loss=8
Época 185/200 [Train]	p=0.289	=2.70e-01: 100%	577/577 [00:20<00:00, 27.67it/s, loss=8

Época 185/200 | p: 0.289 | : 2.70e-01
Train - Total: 85.2524 | MSE: 46.5205 | L0: 181.4
Val - Total: 87.3228 | MSE: 48.8449 | L0: 180.5

Época 186/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.58it/s, loss=8
 Época 187/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.66it/s, loss=8
 Época 188/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.71it/s, loss=8
 Época 189/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.63it/s, loss=8
 Época 190/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.60it/s, loss=8

Época 190/200 | p: 0.289 | : 2.70e-01
 Train - Total: 85.2766 | MSE: 46.7299 | L0: 179.8
 Val - Total: 87.3568 | MSE: 49.1685 | L0: 178.6

Época 191/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.61it/s, loss=8
 Época 192/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.65it/s, loss=8
 Época 193/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.66it/s, loss=8
 Época 194/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.58it/s, loss=8
 Época 195/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.70it/s, loss=8

Época 195/200 | p: 0.289 | : 2.70e-01
 Train - Total: 85.2390 | MSE: 46.6433 | L0: 179.6
 Val - Total: 87.4108 | MSE: 49.0606 | L0: 178.7

Época 196/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.57it/s, loss=8
 Época 197/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.78it/s, loss=8
 Época 198/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.74it/s, loss=8
 Época 199/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.67it/s, loss=8
 Época 200/200 [Train] p=0.289 =2.70e-01: 100%| | 577/577 [00:20<00:00, 27.61it/s, loss=8

Época 200/200 | p: 0.289 | : 2.70e-01
 Train - Total: 85.1711 | MSE: 46.5507 | L0: 179.4
 Val - Total: 87.2768 | MSE: 48.9367 | L0: 178.4

=====
 RESUMEN DEL ENTRENAMIENTO
 =====
 Entrenamiento completo: 200 épocas

Métricas Finales (Última Época):
 Métrica Train Validation
 Total Loss 85.1711 87.2768

MSE Loss	46.5507	48.9367
L0 Sparsity	179.4	178.4
Fraction Active	2.19%	2.18%

```
import torch
ck = torch.load('saved_model/sae_standard_p-annealing_l6_12500g_final.pt', weights_only=False)
print(ck['history']['p_history'])
```

[illegible]

8. Guardar Modelo

```
# Guardar checkpoint final (local + organizado)
checkpoint = {
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'config': CONFIG,
    'history': history
}

# Guardado local
final_model_name = get_model_name(
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],
    estado='final'
)

checkpoint_path = CONFIG['save_dir'] / final_model_name
torch.save(checkpoint, checkpoint_path)

# Guardado organizado
save_checkpoint(
    model,
    base_path=NAMING_META['base_path'],
    variante=NAMING_META['variante'],
    tecnica=NAMING_META['tecnica'],
    capa=NAMING_META['capa'],
    juegos=NAMING_META['juegos'],

```



```

    estado='final',
    extras=NAMING_META['extras'],
    config=CONFIG,
    history=history,
    optimizer=optimizer
)

# Resumen de archivos guardados
saved_files = pd.DataFrame({
    'Tipo': ['Local', 'Organizado'],
    'Ubicación': [
        str(checkpoint_path),
        f"layer_{NAMING_META['capa']:02d}/models/..."
    ]
})

print('\n' + '='*70)
print(' CHECKPOINTS GUARDADOS')
print('='*70)
print(saved_files.to_string(index=False))
print('='*70)

```

Checkpoint guardado: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_standard\25000games\sae_standard_p-annealing_l6_25000g_ex1_final.pt

```

=====
CHECKPOINTS GUARDADOS
=====

```

Tipo	Ubicación
Local	saved_model\sae_standard_p-annealing_l6_25000g_final.pt
Organizado	layer_06/models/...

```

=====

```

9. Métricas básicas del modelo

```

# Guardar metricas de entrenamiento
metrics_dir = get_metrics_dir(
    NAMING_META["experiment_base_path"],
    NAMING_META["variante"],
    NAMING_META["tecnica"],

```

```

NAMING_META["capa"],
NAMING_META["juegos"],
extras_id=extras_id
)

training_metrics = {
    # Metricas de calidad
    "val_mse_final": history["val_mse_loss"][-1],
    "val_mse_best": best_val_mse,
    "val_l0_final": history["val_l0_sparsity"][-1],
    "val_fraction_active_final": history["val_fraction_active"][-1],
    # Contexto del entrenamiento
    "best_epoch": best_epoch,
    "total_epochs": len(history["train_total_loss"]),
    "train_mse_final": history["train_mse_loss"][-1],
    "train_l0_final": history["train_l0_sparsity"][-1],
    # Hiperparametros
    "hidden_dim": CONFIG["hidden_dim"],
    "expansion_factor": CONFIG["hidden_dim"] / CONFIG["input_dim"],
    "sparsity_coef": CONFIG["sparsity_coef"],
    "p_start": CONFIG["p_start"],
    "p_end": CONFIG["p_end"],
    "n_sparsity_updates": CONFIG["n_sparsity_updates"],
    "anneal_start": CONFIG["anneal_start"],
    "learning_rate": CONFIG["learning_rate"],
    "num_games": NAMING_META["juegos"],
}

metrics_path = metrics_dir / "training_metrics.json"
with open(metrics_path, "w") as f:
    _json.dump(training_metrics, f, indent=2)

metrics_df = pd.DataFrame({
    "Metrica": ["Val MSE (final)", "Val MSE (mejor)", "Val L0 (final)", "Fraccion Activa"],
    "Valor": [
        f"{training_metrics['val_mse_final']:.6f}",
        f"{training_metrics['val_mse_best']:.6f}",
        f"{training_metrics['val_l0_final']:.1f}",
        f"{training_metrics['val_fraction_active_final']:.2%}"
    ]
})
print("" + "="*50)

```

```

print(" METRICAS DE ENTRENAMIENTO GUARDADAS")
print("="*50)
print(metrics_df.to_string(index=False))
print(f"Archivo: {metrics_path}")
print("="*50)

```

```

=====
METRICAS DE ENTRENAMIENTO GUARDADAS
=====

```

	Metrica	Valor
Val MSE (final)		48.936732
Val MSE (mejor)		inf
Val LO (final)		178.4
Fraccion Activa		2.18%
Archivo:	c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\annealing_standard\25000games\ex1\metrics\training_metrics.json	

```

=====

```

10. Visualización de Resultados

```

# Gráficas de pérdidas (Train vs Validation) + P-annealing schedule
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# Total loss
axes[0, 0].plot(history['train_total_loss'], label='Train', alpha=0.8)
axes[0, 0].plot(history['val_total_loss'], label='Validation', alpha=0.8)
axes[0, 0].set_title('Total Loss')
axes[0, 0].set_xlabel('Época')
axes[0, 0].set_ylabel('Loss')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# MSE loss
axes[0, 1].plot(history['train_mse_loss'], label='Train', color='orange', alpha=0.8)
axes[0, 1].plot(history['val_mse_loss'], label='Validation', color='red', alpha=0.8)
if CONFIG['early_stopping'] and best_epoch > 0:
    axes[0, 1].axvline(best_epoch - 1, color='green', linestyle='--', alpha=0.7, label=f'Best')
axes[0, 1].set_title('MSE Loss (Reconstrucción)')
axes[0, 1].set_xlabel('Época')
axes[0, 1].set_ylabel('MSE')

```

```

axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# L0 sparsity
axes[0, 2].plot(history['train_l0_sparsity'], label='Train', color='green', alpha=0.8)
axes[0, 2].plot(history['val_l0_sparsity'], label='Validation', color='darkgreen', alpha=0.8)
axes[0, 2].set_title('L0 Sparsity (Features Activas)')
axes[0, 2].set_xlabel('Época')
axes[0, 2].set_ylabel('Número de features')
axes[0, 2].legend()
axes[0, 2].grid(True, alpha=0.3)

# Fraction active
axes[1, 0].plot(history['train_fraction_active'], label='Train', color='purple', alpha=0.8)
axes[1, 0].plot(history['val_fraction_active'], label='Validation', color='magenta', alpha=0.8)
axes[1, 0].set_title('Fracción de Features Activas')
axes[1, 0].set_xlabel('Época')
axes[1, 0].set_ylabel('Fracción')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# P schedule (annealing)
axes[1, 1].plot(history['p_history'], color='brown', linewidth=2, label='p')
axes[1, 1].set_title('P Schedule (Annealing)')
axes[1, 1].set_xlabel('Época')
axes[1, 1].set_ylabel('p')
axes[1, 1].set_ylim(0, 1.1)
axes[1, 1].grid(True, alpha=0.3)

# Train vs Val MSE Gap
mse_gap = [v - t for v, t in zip(history['val_mse_loss'], history['train_mse_loss'])]
axes[1, 2].plot(mse_gap, color='crimson', linewidth=2)
axes[1, 2].axhline(0, color='black', linestyle='--', alpha=0.5)
if CONFIG['early_stopping'] and best_epoch > 0:
    axes[1, 2].axvline(best_epoch - 1, color='green', linestyle='--', alpha=0.7, label=f'Best Epoch')
axes[1, 2].set_title('Overfitting Gap (Val MSE - Train MSE)')
axes[1, 2].set_xlabel('Época')
axes[1, 2].set_ylabel('Gap MSE')
axes[1, 2].legend()
axes[1, 2].grid(True, alpha=0.3)

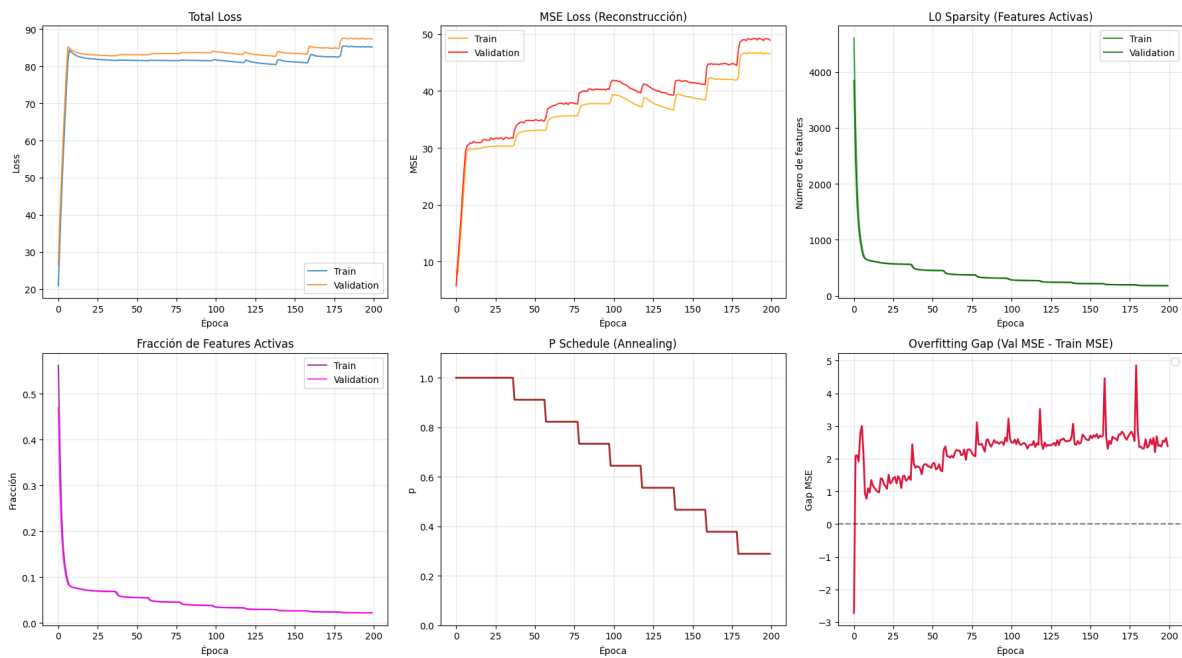
plt.tight_layout()

```

```
plt.savefig(CONFIG['plots_dir'] / 'training_curves.png', dpi=300, bbox_inches='tight')
plt.show()

print('\n' + '='*50)
print('  Gráficas guardadas exitosamente')
print('='*50)
```

C:\Users\Esposa\AppData\Local\Temp\ipykernel_11716\2801488273.py:59: UserWarning: No artists axes[1, 2].legend()



Gráficas guardadas exitosamente

11. Resumen Final

```
# Resumen final completo
print('\n' + '='*70)
print(' RESUMEN FINAL DEL ENTRENAMIENTO')
```

```

print('='*70)

# Arquitectura
arch_summary = pd.DataFrame({
    'Componente': ['Input', 'Hidden', 'Parámetros'],
    'Valor': [
        f"{CONFIG['input_dim']}",
        f"{CONFIG['hidden_dim']} ({CONFIG['hidden_dim']/CONFIG['input_dim']:.1f}x)",
        f"{num_params:}"
    ]
})
print('\nArquitectura:')
print(arch_summary.to_string(index=False))

# Datos
data_summary = pd.DataFrame({
    'Conjunto': ['Train', 'Validation', 'Total'],
    'Muestras': [
        f"{len(train_dataset):}",
        f"{len(val_dataset):}",
        f"{len(full_dataset):}"
    ]
})
print('\nDatos:')
print(data_summary.to_string(index=False))

# Métricas finales
final_summary = pd.DataFrame({
    'Métrica': ['Train Loss', 'Val Loss', 'Train MSE', 'Val MSE', 'Train L0', 'Val L0'],
    'Valor': [
        f"{history['train_total_loss'][-1]:.4f}",
        f"{history['val_total_loss'][-1]:.4f}",
        f"{history['train_mse_loss'][-1]:.4f}",
        f"{history['val_mse_loss'][-1]:.4f}",
        f"{history['train_l0_sparsity'][-1]:.1f}",
        f"{history['val_l0_sparsity'][-1]:.1f}"
    ]
})
print('\nMétricas Finales:')
print(final_summary.to_string(index=False))

# Archivos

```

```

files_summary = pd.DataFrame({
    'Tipo': ['Modelo', 'Gráficas'],
    'Ubicación': [
        str(checkpoint_path),
        str(CONFIG['plots_dir'] / 'training_curves.png')
    ]
})
print('\nArchivos Guardados:')
print(files_summary.to_string(index=False))

print('\n' + '='*70)
print('  ENTRENAMIENTO COMPLETADO EXITOSAMENTE')
print('='*70)

```

```

=====
RESUMEN FINAL DEL ENTRENAMIENTO
=====

```

Arquitectura:

Componente	Valor
Input	512
Hidden	8192 (16.0x)
Parámetros	8,397,312

Datos:

Conjunto	Muestras
Train	1,180,000
Validation	295,000
Total	1,475,000

Métricas Finales:

Métrica	Valor
Train Loss	85.1711
Val Loss	87.2768
Train MSE	46.5507
Val MSE	48.9367
Train L0	179.4
Val L0	178.4

Archivos Guardados:

Tipo

```
Modelo
annealing_l6_25000g_final.pt
Gráficas c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard
annealing_standard\25000games\ex1\plots\training_curves.png
```

```
=====
ENTRENAMIENTO COMPLETADO EXITOSAMENTE
=====
```

12. Generar PDF con Quarto

Una vez ejecutado todo el notebook, ejecutar el siguiente comando en la terminal para generar el PDF con Quarto:

```
# Obtener directorio del experimento y nombre del PDF
exp_dir = get_experiment_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

pdf_name = get_report_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'training',
    extras_id=extras_id
)

notebook_name = 'train_sae_standard_pannealing.ipynb'
output_path = exp_dir / pdf_name

print(f' Generando PDF con Quarto...')
print(f' Archivo de salida: {output_path}')

# Quarto no acepta rutas en --output, solo nombre de archivo
# Se usa --output-dir para el directorio y --output solo para el nombre
result = subprocess.run(
```



```

    f'quarto render {notebook_name} --to pdf --output-dir "{exp_dir}" --output {pdf_name}',
    shell=True,
    capture_output=True,
    text=True
)

if result.returncode == 0:
    print(f' PDF generado exitosamente: {output_path}')
else:
    print(f' Error al generar PDF:')
    print(result.stderr)

```

Generando PDF con Quarto...

Archivo de salida: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard_p-annealing_16_25000games\ex1\sae_standard_p-annealing_16_25000g_ex1_training.pdf

PDF generado exitosamente: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard_p-annealing_16_25000games\ex1\sae_standard_p-annealing_16_25000g_ex1_training.pdf